Para probar tu front-end con un simulador MQTT, te recomiendo utilizar Mosquitto como broker y un pequeño publicador en Python que simule las temperaturas de los 20 super freezers. Así vas a poder comprobar las temperaturas en tiempo real, los cambios de color, las alarmas y los estados sin necesitar sensores físicos.

Hay un detalle importante en tus archivos actuales: `config.js` tiene `DEMO: true`. En ese modo, el front-end genera sus propias temperaturas ficticias y no se conecta a Mosquitto. Para utilizar el broker, tendremos que cambiarlo a `false` y configurar MQTT sobre WebSocket.

1. Arquitectura que vamos a configurar

Simulador Python

Publica 20 equipos, sus estados y temperaturas

Mosquitto

MQTT TCP: 1883 · WebSocket: 9001

Tu front-end

HTML + CSS + JavaScript en el navegador

Los puertos tienen funciones distintas:

- 1883: para que Python publique y reciba mensajes MQTT.
- 9001: para que JavaScript se conecte al broker mediante WebSocket.
- HTTP: para servir la página web. Es independiente de MQTT; por ejemplo, puede ser el puerto 8000 durante las pruebas.

1. Qué necesitás instalar

Mosquitto

El broker MQTT que recibirá los datos del simulador y los distribuirá al navegador.

Descarga oficial: mosquitto.org/download

Python

Para ejecutar un pequeño simulador que publicará temperaturas automáticamente.

Descarga oficial: python.org/downloads/windows

No necesitás instalar Node.js ni montar todavía el back-end completo. Tu `app.js` ya carga MQTT.js desde una CDN y puede comunicarse directamente con Mosquitto mediante WebSocket.

2. Instalar Mosquitto en Windows

1. Descargá el instalador de Windows desde el sitio oficial de Mosquitto.
2. Ejecutalo y, si el instalador ofrece la opción, incluí los componentes de cliente (`mosquitto_pub` y `mosquitto_sub`).
3. Si el instalador solicita instalar un componente adicional como dependencia, seguí las indicaciones que aparezcan.
4. Comprobá dónde quedó instalado. La ubicación habitual es:

C:\Program Files\mosquitto\

Abrí PowerShell y ejecutá:

Test-Path "C:\Program Files\mosquitto\mosquitto.exe"

Si devuelve `True`, el ejecutable está en esa ubicación. Si devuelve `False`, localizá la carpeta en la que lo instalaste.

También podés comprobar la versión con:

& "C:\Program Files\mosquitto\mosquitto.exe" -h

3. Configurar los puertos MQTT y WebSocket

El navegador necesita el puerto `9001` para conectarse por WebSocket. Vamos a crear una configuración propia para las pruebas locales.

Abrí el Bloc de notas y pegá este contenido:

```
# Broker MQTT local para pruebas
listener 1883 127.0.0.1
protocol mqtt

# WebSocket para el navegador
listener 9001 127.0.0.1
protocol websockets

# Solo para pruebas locales
allow_anonymous true

# Logs en la consola
log_dest stdout
log_type all
```

Guardalo como:

C:\freezers-mqtt\mosquitto.conf

Para crear esa carpeta, podés ejecutar en PowerShell:

```
New-Item -ItemType Directory -Force C:\freezers-mqtt
notepad C:\freezers-mqtt\mosquitto.conf
```

Pegá el contenido, guardá y cerrá el Bloc de notas. Asegurate de que el archivo se llame `mosquitto.conf` y no `mosquitto.conf.txt`.

Iniciar el broker

En PowerShell ejecutá:

```
& "C:\Program Files\mosquitto\mosquitto.exe" -c "C:\freezers-mqtt\mosquitto.conf" -v
```

Dejá esa ventana abierta. Si los puertos están libres y la configuración es correcta, Mosquitto debería indicar que escucha en `1883` y `9001`.

Si aparece un error de puerto ocupado, puede que el servicio de Mosquitto instalado en Windows ya esté ejecutándose. En ese caso, revisá `services.msc` y detené temporalmente el servicio antes de iniciar esta instancia de pruebas.

4. Comprobar que MQTT funciona

Abrí otra ventana de PowerShell y suscribite a los mensajes:

```
& "C:\Program Files\mosquitto\mosquitto_sub.exe" -h 127.0.0.1 -p 1883 -t "freezers/#" -v
```

Abrí una tercera ventana de PowerShell y publicá una temperatura:

```
& "C:\Program Files\mosquitto\mosquitto_pub.exe" -h 127.0.0.1 -p 1883 -t
"freezers/eq01/temperatura" -m "-79.5"
```

En la segunda ventana debería aparecer:

```
freezers/eq01/temperatura -79.5
```

Si ocurre, la comunicación MQTT básica está funcionando.

5. Adaptar tu `config.js`

En la carpeta de los cuatro archivos, abrí `config.js` y cambiá la configuración inicial a estos valores:

```
const CONFIG = {
  DEMO: false,

  MQTT_URL: "ws://127.0.0.1:9001",
  MQTT_USER: "",
```

```
  MQTT_PASS: "",
  TOPIC_BASE: "freezers",

  API_EQUIPOS: "",
  API_HISTORIAL: "/api/historial/{id}",

  UMBRAL_ALARMA: -70,
  MARGEN_AVISO: 5,
  STALE_MS: 120000,
};
```

Mantené el resto de los comentarios si querés; lo importante es que los valores sean correctos.

No cambies `app.js` ni `styles.css` por ahora. El código que compartiste ya reconoce los temas MQTT necesarios.

6. Abrir el front-end en Windows

Supongamos que tus cuatro archivos están guardados en:

```
C:\freezers-web\
    index.html
    styles.css
    config.js
    app.js
```

Abrí PowerShell y ejecutá:

```
cd C:\freezers-web
py -m http.server 8000
```

Si `py` no se reconoce, necesitás instalar Python o corregir su instalación.

Abrí el navegador en:

http://127.0.0.1:8000

No abras `index.html` directamente haciendo doble clic: para esta prueba es mejor servirlo mediante HTTP.

En la barra superior, el estado debería cambiar a «Conectado» si Mosquitto está escuchando en el puerto `9001` y el WebSocket funciona.

7. Enviar un equipo de prueba

Con el broker y la página abiertos, ejecutá estos comandos en PowerShell, uno por uno:

```
& "C:\Program Files\mosquitto\mosquitto_pub.exe" -h 127.0.0.1 -p 1883 -t "freezers/eq01/info" -m '{"numero":60,"grupo":"Pérez","piso":3}' -r
& "C:\Program Files\mosquitto\mosquitto_pub.exe" -h 127.0.0.1 -p 1883 -t "freezers/eq01/estado" -m "funcionando" -r
& "C:\Program Files\mosquitto\mosquitto_pub.exe" -h 127.0.0.1 -p 1883 -t "freezers/eq01/temperatura" -m "-79.5"
```

Deberías ver el equipo 60, del grupo Pérez, piso 3, con una temperatura verde.

Ahora probá la alarma:

```
& "C:\Program Files\mosquitto\mosquitto_pub.exe" -h 127.0.0.1 -p 1883 -t "freezers/eq01/temperatura" -m "-69"
```

La fila debería ponerse roja e intermitente. Para comprobar el sonido, activá el botón de sonido de la página y verificá que el navegador no esté silenciado.

Si el broker se conecta pero el equipo no aparece, abrí las herramientas de desarrollador con `F12` y revisá Console. Si no conecta, revisá primero los mensajes de la terminal donde ejecutaste Mosquitto.

8. Qué falta para completar la simulación

Con estos pasos podés probar manualmente un equipo. Para simular los 20 equipos automáticamente, el siguiente paso es crear un script Python que publique:

- Los 20 equipos y sus datos identificativos.
- Temperaturas nuevas cada dos segundos.
- Dos equipos con estado `no funcionando`.
- Un equipo que se acerque al umbral y otro que lo supere.
- Mensajes retenidos para la información y el estado.

El historial todavía necesitará un backend Python con almacenamiento y una ruta HTTP para recuperar las mediciones.